

福州大学

## 本科生毕业设计（论文）

题    目： 面向亿门级 FPGA 电路的划分算法

研究

姓    名： 张驭龙

学    号： 032201115

学    院： 物理与信息工程学院

专    业： 信息与计算科学

年    级： 2022 级

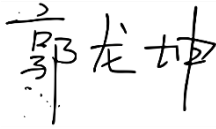
校内指导教师： 郭龙坤 （签名）

校外指导教师： \_\_\_\_\_ （签名）

2026 年 5 月 31 日



福州大学本科生毕业设计（论文）诚信承诺书

|                                                                                                                                                                                                                                                                            |           |                                                             |      |      |           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------|------|------|-----------|
| 学生姓名                                                                                                                                                                                                                                                                       | 张驭龙       | 年 级                                                         | 2022 | 学 号  | 032201115 |
| 所在学院                                                                                                                                                                                                                                                                       | 物理与信息工程学院 |                                                             |      | 所学专业 | 信息与计算科学   |
| 毕业设计（论文）题目                                                                                                                                                                                                                                                                 |           | 中文：面向亿门级 FPGA 电路的划分算法研究                                     |      |      |           |
|                                                                                                                                                                                                                                                                            |           | 外文：Research of Partitioning Algorithm for Billion-gate FPGA |      |      |           |
| <div>学生承诺</div> <p>我承诺在毕业设计（论文）过程中遵守学校有关规定，恪守学术规范，未存在买卖、代写、作假等行为。在本人的毕业设计（论文）中未剽窃、抄袭他人的学术观点、思想和成果，未篡改实验数据。如有违规行为发生我愿承担一切责任，接受学校的处理。</p> <div>学生（签名）：</div> <div>2026 年 5 月 31 日</div> |           |                                                             |      |      |           |
| <div>指导教师承诺</div> <p>我承诺在指导学生毕业设计（论文）过程中遵守学校有关规定，恪守学术规范，经过本人认真的核查，该同学未存在买卖、代写、作假等行为，毕业设计（论文）中未发现剽窃、抄袭他人的学术观点、思想和成果的现象，未发现篡改实验数据。</p> <div>指导教师（签名）：</div> <div>2026 年 5 月 31 日</div>  |           |                                                             |      |      |           |



# 面向亿门级 FPGA 电路的划分算法研究

## 摘要

现场可编程逻辑门阵列（Field Programmable Gate Array, FPGA）是作为专用集成电路领域中的一种半定制电路而出现的，因其具有更高的集成度、更强的逻辑功能和更大的灵活性，目前已成为设计数字电路或系统的首选器件之一。正因如此，对与 FPGA 相关的研发、设计技术的需求与日俱增，其中便包括面向亿门级的多芯片 FPGA 电路的划分算法的研究。

面向亿门级的多芯片 FPGA 电路的划分算法的基础是超图划分技术。国内外在超图划分领域的研究成果已十分丰富，从而产生了许多可行且有效的多芯片 FPGA 电路的划分算法。未来随着科技的发展与人类的需要，FPGA 电路将会更加集成，为顺应未来发展趋势，本课题便是在国内外研究成果的基础上实现将现行可用的划分算法更新迭代为可完成大规模划分任务的可行且有效的算法。

该算法总共分为三步：准备工作、初始划分、优化方案。其中准备工作为将所需数据数字化并生成相应写有超图信息的文件用于第二步操作；初始划分为使用 Catalyurek U 等人提供的闭源可运行的 PaToH（Partitioning Tool for Hypergraph）对数据进行划分得到初始划分结果；优化方案为对第二步生成的初始划分结果基于不同芯片内部资源利用率的平均差异百分比和各个芯片间的物理连线条数的限制，以贪婪放置（Greedy Placement）的思路进行优化，最后得出符合条件的划分结果。

通过将某数据划分到 Xilinx VU13P (4-dies) 的实践任务，验证了该算法在万门级多芯片 FPGA 电路的划分任务方面的可行性和有效性，并基于该实践提出不足与对应解决思路，推理分析该算法在亿门级划分任务中的可行性。

**关键词：**FPGA，划分，C++，集成电路，组合优化



# **Research of Partitioning Algorithm for Billion-gate FPGA**

## **Abstract**

Field Programmable Gate Array (FPGA) is created as a semi-custom circuit in the field of application-specific integrated circuits, because of its higher integration, stronger logical functions and greater flexibility, it has been one of the preferred devices for designing digital circuits or systems. Since the reason mentioned above, there is an increasing demand for R&D (Research and Development) and design technologies related to FPGAs, including the research of partitioning algorithm for billion-gate FPGA.

The theoretical basic of the partitioning algorithm for billion-gate FPGA is the hypergraph partitioning. From home and abroad, there has been many achievements in the field of hypergraph partitioning, which has led to the development of many feasible and effective partitioning algorithms for multi-chip FPGA circuits. In the future, with the development of technology and human's needs, FPGA will be more integrated, to adapt to this trends, this research is going to update the current available partitioning algorithm into a feasible and effective algorithm capable of handling large-scale partitioning tasks.

This algorithm contains three steps: preparation, first-time-partitioning, optimization. Preparation is to digitalize the data we needed and make a file used in step two, containing the information of the hypergraph. First-time-partitioning is to get the first-time partitioning result by using the closed-source and executable PaToH (Partitioning Tool for Hypergraph) provided by Catalyurek U et al. Optimization is to optimize the first-time partitioning result considering the average percentage difference in internal resource utilization across different chips and the limitations on the number of physical inter-chip connections (SLL) using Greedy Placement approach.

By applying the algorithm to the partitioning task base on Xilinx VU13P (4-dies), its feasibility and effectiveness in partitioning tasks for million-gate multi-chip FPGA circuits were verified. Based on this practical application, shortcomings were identified along with corresponding solutions, and the feasibility of the algorithm for billion-gate partitioning tasks was analyzed through reasoning.

**Keywords: FPGA; partitioning; C++; integrated circuit;**

## **combinatorial optimization**



# 目录

|                                                |     |
|------------------------------------------------|-----|
| 摘要 .....                                       | I   |
| <b>Abstract</b> .....                          | III |
| 目录 .....                                       | i   |
| <b>第一章 绪论</b> .....                            | 1   |
| 1.1    引言 .....                                | 1   |
| 1.2    研究现状 .....                              | 1   |
| 1.3    FPGA 电路的划分问题定义 .....                    | 4   |
| 1.4    课题目标 .....                              | 5   |
| <b>第二章 算法设计与研究</b> .....                       | 7   |
| 2.1.    算法流程设计 .....                           | 7   |
| 2.2.    数据结构 .....                             | 8   |
| 2.2.1    数据文件构成 .....                          | 8   |
| 2.2.2    数据结构设计 .....                          | 8   |
| 2.3.    初始划分 .....                             | 9   |
| 2.3.1    划分原理（超图划分理论） .....                    | 9   |
| 2.3.2    PaToH 划分器的使用 .....                    | 10  |
| 2.4.    优化算法 .....                             | 11  |
| 2.4.1    约束条件（划分阈值） .....                      | 11  |
| 2.4.2    贪婪放置（Greedy Placement） .....          | 13  |
| 2.4.3    基于不同芯片内部资源利用率平均差异的阈值的优化 .....         | 13  |
| 2.4.4    基于各个芯片间物理连线的阈值的优化（以“链式”数据流设计为例） ..... | 16  |
| <b>第三章 算法实践检验</b> .....                        | 18  |
| 3.1.    数据数字化 .....                            | 19  |
| 3.2.    初始划分与约束相关参数 .....                      | 19  |
| 3.3.    优化与结果 .....                            | 20  |

|                    |    |
|--------------------|----|
| 第四章 总结 .....       | 22 |
| 4.1.    课题总结 ..... | 22 |
| 4.2.    未来展望 ..... | 22 |
| 附件 .....           | 24 |
| 参考文献 .....         | 25 |
| 附录 1 .....         | 28 |
| 致谢 .....           | 48 |

# 第一章 绪论

## 1.1 引言

现场可编程逻辑门阵列（Field Programmable Gate Array, 下称 FPGA）是一种可反复编程的硬件芯片。它内含大量可配置逻辑单元和可编程连线，用户能通过特定的硬件描述语言定制专用的电路。相较于 CPU 的指令的串行执行，FPGA 凭借硬件的并行的特性实现低延迟、高吞吐，且功耗可控的性能，这种“硬件可重构”的能力使得其被广泛地应用于诸如 5G 通信、图像处理、金融加速、高速数据采集等等多个场景中。FPGA 电路不仅解决了定制电路的不足，还克服了原有可编程器件门电路数有限的缺点，因其具有更高的集成度、更强的逻辑功能和更大的灵活性，目前已成为设计数字电路或系统的首选器件之一。

近年来，FPGA 产业在 AI 与边缘计算浪潮推动下实现了快速增长<sup>[1]</sup>。全球市场规模已从 2025 年的约 125 亿美元攀升至 2026 年的 143 亿美元，年复合增长率高达 14.3%。市场高速扩张的背后，是各行业“硬件可重构”的能力的需求正从传统数字系统设计全面转向 AI/ML 硬件加速、自动驾驶 ADAS 以及 5G/6G 通信基础设施的设计。

在技术层面，FPGA 电路正在经历从单一可编程逻辑器件向异构计算平台的转型。厂商纷纷推出集成 AI Tensor Block、DSP 及嵌入式处理器内核的自适应 SoC（如 Chiplet 级异构集成），在保持灵活性的同时大幅提升算力效率<sup>[20][21]</sup>。硬件可信根、后量子密码等先进的安全特性也被深度集成，从而满足一些高安全场景的应用需求。

在应用层面，FPGA 已从传统的原型验证拓展转变为数据中心智能网卡与 AI 推理加速的关键器件<sup>[22][23]</sup>。在 AI 大模型训练中，FPGA 可用于梯度压缩与通信加速，缓解 GPU 集群的带宽瓶颈；在边缘侧，其确定性的低延迟实时处理能力在机器人与医学成像等领域得到广泛验证。在 5G/6G 通信领域<sup>[19]</sup>，FPGA 更是基站前传接口与核心网设备的理想选择。

因此，对与 FPGA 相关的研发、设计技术的需求与日俱增。

## 1.2 研究现状

与 FPGA 相关的研发、设计技术包括 FPGA 的布局技术和 FPGA 的布线技术等。

其中，有关 FPGA 的布局技术，目前已存在许多种不同的方法策略，主要可以分

为三种：基于划分的布局技术、基于启发式的布局技术以及基于解析式的布局技术<sup>[1]</sup>。

基于划分的布局技术是指根据已知的布局约束关系，在最初始的布局方案的基础上，通过递归迭代来对布局方案进行划分，直至得到的布局方案小于事先设定的阈值为止<sup>[1][5]</sup>的 FPGA 布局技术。以 Kim 等人研究设计的 SimPL 布局方法<sup>[31]</sup>为代表，该方法以线长作为约束条件，分别计算线长的下界与上界，然后通过一种基于自上向下的几何分割和非线性缩放的前向合法化方法不断进行迭代，最终使逻辑单元收敛到合法的位置。基于划分的布局技术的优点在于其将复杂的电路设计分割为较简单的电路设计，从而能够快速对局部电路进行处理，并且其收敛速度快。

基于启发式的布局技术是指以一些自然现象为设计原理，对 FPGA 逻辑单元的布局过程进行数学建模<sup>[1]</sup>的 FPGA 布局技术。在这方面的典型代表有 SADEGHI A 等人所研究的遗传算法<sup>[24]</sup>、HERATH K 等人所研究的蚁群算法<sup>[25]</sup>以及模拟退火算法<sup>[26][27][28]</sup>，其中又以模拟退火算法为基于启发式的布局技术的研究的热门方向。1953 年，Metropolis 等人<sup>[27]</sup>最早提出了模拟退火算法理论，它是一种基于 Monte-Carlo 求解方式的随机搜索算法，其原理是物理中固体物质的退火过程与一般组合优化问题之间的具有一定的相似性，故可通过模拟固体物质的退火过程来解决部分组合优化问题。到了 1988 年，Carl 等人<sup>[28]</sup>首次将模拟退火算法的思想引入到超大规模集成电路（Very Large Scale Integration, VLSI）布局领域。随后，越来越多的研究学者开始关注模拟退火算法在 VLSI、FPGA 等布局领域的应用。目前学界较为流行的 FPGA 通用布局布线（Versatile Placement and Routing, VPR）工具的布局器的基础框架正是模拟退火算法。基于启发式的布局技术的优点在于其相对于最优算法，在时间允许的情况下，能够获得布局的最优解。

基于解析式的布局技术是指将 FPGA 布局问题建模为一个连续优化问题，通过梯度下降的方法得到布局问题的最优解<sup>[1]</sup>的 FPGA 布局技术。以目标函数为分类标准，该布局方法共分为两种：GOPALAKRISHNAN P 等人所研究使用的二次方布局技术<sup>[29]</sup>、LIN T H 等人所研究使用的非线性布局技术<sup>[30]</sup>。二次方布局技术通常以线长为优化目标完成对目标函数的建模，随后通过 Krylov 子空间方法将上述优化问题转化为对称与正定（Symmetric and Positive-Definite, SPD）线性系统的求解问题，最终得到优化问题的解；非线性布局技术则使用不可微分的非线性函数来表示非重叠的约束条件，并将其与线长一起在统一的目标函数中进行求解。基于解析式的布局技术的优点在于其完全依赖于数学方法，其最终的布局质量能够通过数学分析来进行证明，具有较高的布局质量，并且与基于划分的布局方法一样收敛速度较快。

多芯片 FPGA 电路的划分技术本质上是对超图的划分技术，因此图划分与超图划分技术的进步促进了多芯片 FPGA 电路的划分技术的不断迭代。在图划分和超图划分领域中已有不少成果<sup>[7]</sup>，如：

1) 2012 年马永刚改进的图划分模型<sup>[10]</sup>。针对图划分应用于分布式并行计算只能表达对称和双向依赖关系的问题, 提出了基于 0-1 规划方法的并行计算图划分模型; 针对标准图划分模型应用于并行计算不能准确表达通信量, 也不能考虑通信延迟和通信量的分布情况对并行性能的影响, 提出了改进的图划分模型; 针对分布式网络控制中的工业网络优化设计问题, 提出了基于非线性 0-1 规划理论对其进行建模; 为了在分布式交通控制中有效的聚合查询处理, 路网数据应该有效的组织和存储, 针对这一问题提出了基于聚类图模型的改进方法;

2) 2018 年李金吉等人的流式处理的异步图处理框架<sup>[12]</sup>。李金吉等人结合了累加迭代计算和单机并行处理技术, 提出流式处理的异步计算模型 ASP。它实现了对磁盘的完全顺序访问, 允许流式的顺序载入结构数据的同时进行异步更新计算。基于 ASP 模型, 又提出了一种流式处理的异步图处理框架 S-Maiter, 实现了高效率的基于外存的单机大图处理, 通过 I/O 线程优化、内存资源监控、shard 级优先级调度等优化技术, 提高了系统处理大图数据的性能;

3) 2023 年李贺等人对动态图划分算法的研究整理<sup>[13]</sup>。其中包括流式图划分算法、增量式图划分算法和图重划分算法。流式图划分算法包括基于 Hash 的划分算法、基于邻居分布的划分算法以及基于流的优化划分算法, 增量式图划分算法包括单元素增量式划分算法和批量增量式划分算法, 图重划分算法包括针对图结构动态的重划分算法和针对图计算动态的重划分算法;

4) 2025 年路文军等人的宽度优先超图划分技术<sup>[8]</sup>。该算法增加一个增益函数使宽度优先算法中的顶点除按照与原始顶点相关性强弱进行优先级排列外, 对相关性强弱相同的顶点根据增益值的大小进行排列生成初始划分结果, 然后借鉴贪心划分算法的思路, 在保证可行性的基础上将部分顶点按照宽度优先的优先级排列划分到两块中, 对于剩余的自由顶点根据增益值进行划分;

5) ÇATALYÜREK Ü 等人<sup>[7]</sup>、尚俊霖等人<sup>[11]</sup>、Fischer E 等人<sup>[14]</sup>以及 Buluç A 等人<sup>[15]</sup>对 2025 年以前的(超)图划分技术的综述。详细(超)图划分理论可参考 2.3.1。

在前述超图划分理论基础的便诞生了多芯片 FPGA 电路的划分算法<sup>[1][3][4]</sup>, 并且出现了许多基于上述理论的实际案例, 如:

1) 2005 年 KHATKHATE A 等人的基于递归二分法的布局技术<sup>[5]</sup>。他们将传统的递归二分标准单元格放置工具扩展到直接考虑混合块设计;

2) 2023 年李本正的多 FPGA 系统结构感知的电路快速划分方法<sup>[9]</sup>。他针对 RTL 电路划分的问题, 提出了一种将 RTL 电路映射为超图的框架; 提出了一种面向 MFS 结构的、高性能的、可以避免 FPGA 间信号中转的电路划分方法; 在避免 FPGA 间信号中转的电路划分方法的基础上, 提出了一种新型的 MFS 感知划分框架; 针对学术与工业界急需一个高效的面向 MFS 大电路划分器的问题, 将提出的划分算法部署

成为了可以实际应用的划分器工具；

3) 2026 年黄宏赞等人的多维感知的多芯片 FPGA 划分方法<sup>[2]</sup>。他们针对跨芯片时序约束和物理拓扑约束的挑战，基于多级范式提出了一种多维约束感知的多芯片 FPGA 网表划分方法，突破通用工具仅平衡分割的局限。

### 1.3 FPGA 电路的划分问题定义

本课题为面向亿门级 FPGA 电路的划分算法研究，即本课题所研究的算法要实现将亿门级的节点数合理地分布在多芯片 FPGA 电路（如 Xilinx VU13P (4-dies)）的芯片上。简单来看，假设节点数为  $n$ ，目标多芯片 FPGA 电路的芯片数为  $k$ ，则该算法便是用来实现将  $n$  个节点根据特定的约束条件划分到  $k$  个芯片中。

正如 1.2 所述，多芯片 FPGA 电路的划分本质上是超图划分。我们假设一个超图  $H = (V, N)$ ，那么我们可以把  $n$  个节点看作超图中的  $V$ 、 $n$  个节点的相互关系所构成的超边看作超图中的  $N$ ，芯片数  $k$  看作我们的划分目标路数  $K$ ，则该算法所实现的功能便是对上述超图  $H$  进行  $K$  路划分。

超图划分相关概念与定义如下：

**定义 1** 无向超图  $H(V, N, c, \omega)$  由一个顶点集  $V$  和一个超边集  $N$  组成，并且顶点的权重为  $c: V \rightarrow R_{\geq 0}$ ，超边的权重为  $\omega: N \rightarrow R_{\geq 0}$ 。取  $v \in V$ ， $n \in N$ ，若  $v \in n$ ，则表示顶点  $v$  与超边  $n$  相邻。若  $\exists n \in N: v \in n \wedge u \in n$ ，则表示顶点  $v$  和  $u$  相邻。记  $I(v) = \{n | n \in N \wedge v \in n\}$  表示与顶点  $v$  相邻的所有超边  $n$  的集合。记  $d(v) = |I(v)|$  表示顶点  $v$  的度。若  $d(v) = 0$ ，则称顶点  $v$  为孤立点。记  $\Gamma(v) = \{u | \exists n \in N: v \in n \wedge u \in n\}$  表示与顶点  $v$  相邻的所有顶点  $u$  的集合。记  $|n|$  表示超边  $n$  的大小。若  $|n| = 1$ ，则称超边  $n$  为单节点超边。若  $n_i = n_j$ ，则称超边  $n_i$  与  $n_j$  是并行的。

**定义 2** 超图  $H$  的  $K$  路划分  $P = \{V_1, V_2, \dots, V_K\}$  是将超图  $H$  的顶点集  $V$  划分成  $K$  个部分，且满足约束条件：

- 1)  $\bigcup_{i=1}^K V_i = V \wedge \forall V' \in P: V' = \emptyset$ ;
- 2)  $\forall i, j \in [1, 2, \dots, K]: i = j \Rightarrow V_i \cap V_j = \emptyset$ ;
- 3)  $\forall V' \in P: c(V') \leq L_{max}$ 。

其中  $L_{max} = \left\lceil \frac{c(V)}{K} \right\rceil (1 + \varepsilon) + \max_{v \in V} c(v)$ ，表示划分  $P$  是满足  $\varepsilon$ -平衡的。

**定义 3** 一个超边  $n$  在块  $V_i$  中的顶点数记为  $\varphi(n, V_i) = |\{v | v \in V_i \wedge v \in n\}|$ 。若  $\varphi(n, V_i) > 0$  则说明超边  $n$  与块  $V_i$  有关。若  $\varphi(n, V_i) = 0$  则说明超边  $n$  与块  $V_i$  无关。超边  $n$  的连通集记为  $\Lambda(n) = \{V_i | \varphi(n, V_i) > 0\}$ 。记  $\lambda(n) = |\Lambda(n)|$  表示超边  $n$  的连通集的大小。若  $\lambda(n) > 1$ ，则称超边  $n$  为切边。若  $\lambda(n) = 1$ ，则称超边  $n$  为内部边。

定义 4 超图  $H$  的划分  $P$  的切边度量的计算公式如下,

$$\kappa(H, P) = \sum_{n \in N \wedge \lambda(n) > 1} \omega(n)$$

超图划分的目标便是最小化切边度量  $\kappa(H, P)$ 。

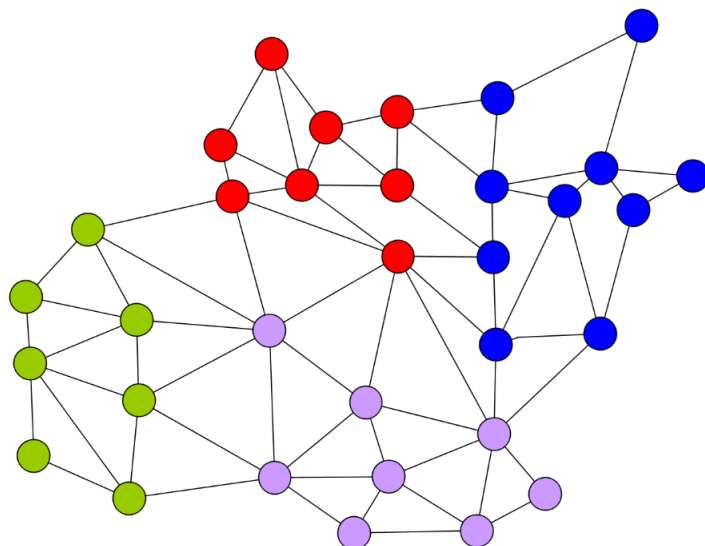


图 1 图划分示意图

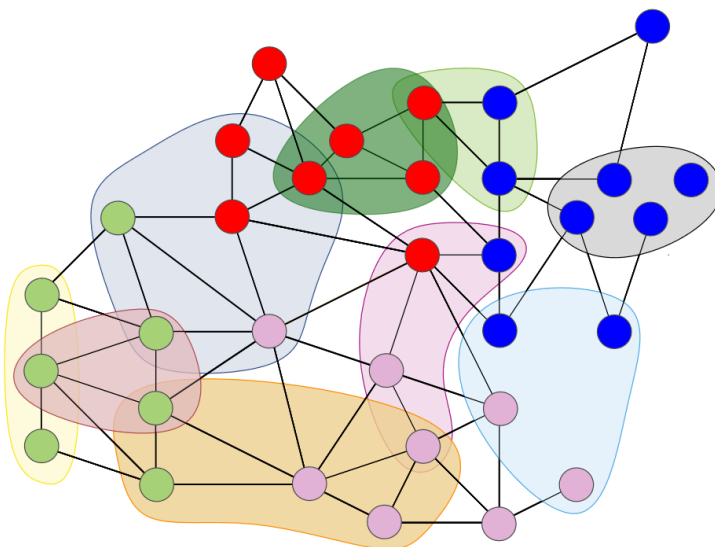


图 2 超图划分示意图

## 1.4 课题目标

本课题旨在在国内外前辈的研究成果的基础上, 探索并设计出一套能实现将亿门级数据划分到多芯片 FPGA 电路的划分算法, 并利用 C++ 代码实现该算法, 通过简单

的实践任务来检验该算法的可行性和有效性。

全文共分为四个部分，分别为，

1) 第一章为绪论。主要内容为本课题的研究背景介绍、国内外与本课题相关的研究成果的收集以及本课题的研究意义与计划。

2) 第二章为算法设计与研究。主要内容为针对本课题的多芯片 FPGA 电路的划分算法的分步设计及其相应理论与逻辑推导。

3) 第三章为实践检验。主要内容为使用第二章设计出的算法完成将某组数据划分到 Xilinx VU13P (4-dies) 的四个芯片上，并分析结果，判断任务是否完成。

4) 第四章为总结。主要内容为基于第三章的实践检验结果判断算法的可行性与有效性以及本课题存在的不足之处与相应解决方向，最后对本课题的未来发展进行展望。



## 第二章 算法设计与研究

### 2.1. 算法流程设计

该算法所实现的功能为：输入一份数据文件，输出对应的划分结构。故可设计出  
具体的实现流程如图 3 所示。

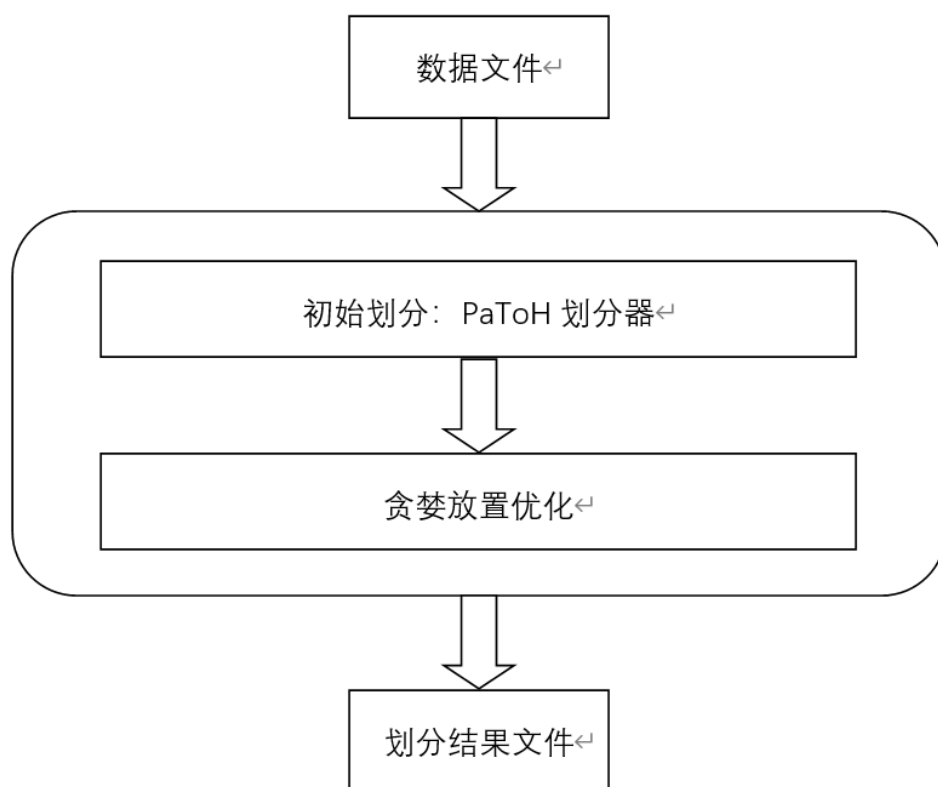


图 3

图 1 中，该算法共分三步走：

- 1) 提取信息。数据文件经过数字化处理后获取所有信息（用于存放数据文件相关信息的数据结构设计详见2.2），并如1.3 所述构建超图文件；
- 2) 初始划分。使用 PaToH 划分器（详见2.3）将超图文件进行初始划分得到初始划分结果；
- 3) 优化。根据所需限制条件（详见2.4），结合贪婪放置的优化思想得到最终项目所需的划分结果。

## 2.2. 数据结构

### 2.2.1 数据文件构成

原始数据文件包含 10 个子文件，其中本课题所用到的文件包括 Object File Library (.lib)、NETS (.nets) 和 NODES (.nodes) 文件，详见附件1。

Object File Library 文件内容为该数据所用资源（cell）以及各个 cell 的引脚（pin）类型与对应的输入输出类型。以图 4 的(a)为例。

NETS 文件内容为该数据的网络（net）信息，包含 net 的名称、net 中的节点（node）以及 node 对应的引脚类型。以图 4 的(b)为例。

NODES 文件内容为该数据的节点（node）信息，包含 node 的名称以及该节点所消耗的资源（cell）。以图 4 的(c)为例。

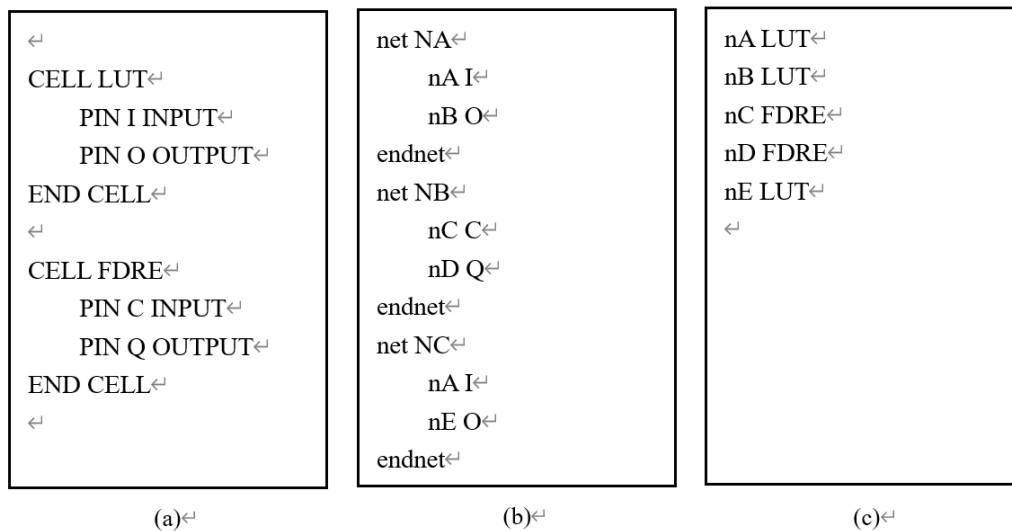


图 4 (a) Object File Library 文件示例，如图中有两种 cell，对应的输入引脚 I 和 C，输出引脚 C 和 Q；(b) NETS 文件示例，如图中有三个 net，其中 NA 有输入节点 nA 和输出节点 nB、NB 有输入节点 nC 和输出节点 nD、NC 有输入节点 nA 和输出节点 nE；(c) NODES 文件示例，如图中有五个节点以及其对应的 cell。

### 2.2.2 数据结构设计

在前述数据文件构成基础上，设计数据结构如下表 1。

| 类 (class) | 变量             | 说明                 |
|-----------|----------------|--------------------|
| cell      | string name    | cell 的名称           |
| die       | int capacity   | die 中的 node 的个数    |
|           | int list[5000] | die 中的 node 的索引    |
| net       | string name    | net 的名称            |
| node      | string name    | node 的名称           |
|           | int cell       | node 所消耗的 cell 的索引 |
|           | int die        | node 所在的 die 的索引   |

表 1

如表 1 所示, 设计了四个类, 分别为 cell, die, net 以及 node。

类 cell 包含的变量有 name (cell 的名称)。

类 die 包含的变量有 capacity (die 中 node 的数量), list (die 中有哪些 node) 以及加入 die 函数和从 die 中删除的函数。

类 net 包含的变量有 name (net 的名称)。

类 node 包含的变量有 name (node 的名称), cell (node 所消耗的 cell) 以及 die (node 所在的 die)。

具体 C++源代码详见附件2 中的 cpp/cell.h, cpp/dies.h, cpp/net.h, cpp/nodes.h 文件。

### 2.3. 初始划分

正如1.2 与1.3 所述, 多芯片 FPGA 电路的划分问题本质上是超图划分问题。我们假设一个超图 $H = (V, N)$ , 那么数据文件中的 node 便是超图中的 $V$ 、net 便是超图中的 $N$ 。如此一来, 我们初始划分的目的, 便是将数据文件中的 node 根据 net 的结构进行一次超图划分操作从而将数据文件中的 node 初步划分到 die 中, 目标 die 有  $K$  个, 我们便将数据文件中的 node 划分成  $K$  个部分。

在对不同划分工具<sup>[6][17]</sup>之间进行对比后, 我选择了由 Catalyurek U 教授和 Aykanat C 教授主导开发的 PaToH (Partitioning Tool for Hypergraph)<sup>[17]</sup>来对数据进行初始划分。

#### 2.3.1 划分原理 (超图划分理论)

超图划分是将超图 $H = (V, N)$ 划分为  $K$  个部分, 即所谓超图  $H$  的  $K$  路划分 ( $k$ -ways partitioning), 其目标是找到一个平衡划分方案使得割 (cut) 的代价 (cost) 越小越好。平衡划分的约束条件为平衡约束, 即每个部分的顶点总权重不超过平均权重的  $1+\epsilon$  倍。

PaToH 划分器采用两种方式计算代价: 一种是切边度量, 即所有被切割超边的代

价之和；一种是连通性度量，即每条被切割超边的代价乘以其连通部分数减 1。

如图 5 所示，超图划分的三个阶段如下，

1) 粗化。将原始超图逐级压缩成越来越小的超图。利用聚类算法将多个顶点合并为一个顶点，并收缩内部超边，逐步减少问题规模，直到顶点数降至阈值，得到最粗的小超图。PaToH 实现了多种聚类算法<sup>[16]</sup>，包括基于匹配的聚类和凝聚式聚类。

2) 初始划分。在最粗的小超图上，运行贪心超图生长算法（Greedy Hypergraph Growth Algorithm）生成初始二分划分结果。该算法从一个随机顶点开始，利用类似 FM 的增益优先队列逐步“生长”出一个满足平衡约束的簇。由于粗图很小，会多次随机运行，选择最优结果进入下一阶段。

3) 细化。将粗超图的二分划分方案逐级映射回更精细的超图。在每一级映射后，都会运行基于 KL/FM（Kernighan-Lin/Fiduccia-Mattheyses）的局部优化算法（如 BFM，即边界 FM）来优化当前的划分方案。这个过程可在多个层级上形成一个“V-循环”进行反复优化，直到恢复原始超图，得到最终的  $K$  路划分结果。

综上所述，PaToH 划分器的划分策略为递归二分法，对于  $K > 2$  的情况，PaToH 划分器采取多次二分超图来实现多路划分。

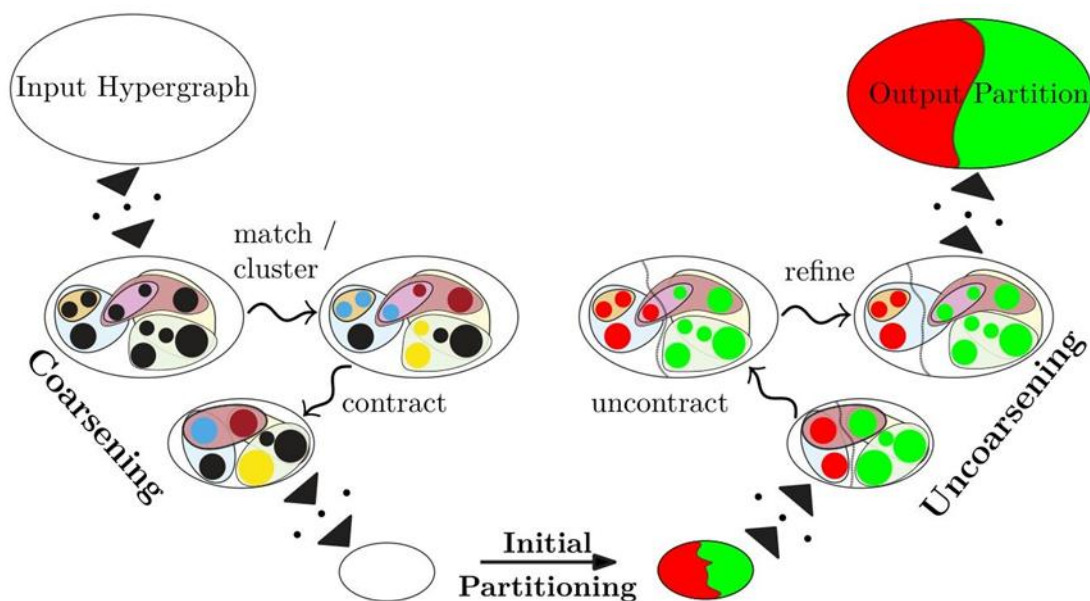


图 5

### 2.3.2 PaToH 划分器的使用

根据 Catalyurek U 教授和 Aykanat C 教授所提供的手册<sup>[17]</sup>，PaToH 划分器的使用方法如下所示。

Input: An HGR (.hgr) file of the hypergraph, the target ways of partitioning K.

Output: A file with a “hgr.part.K” suffix.

即输入为一份后缀为 “.hgr” 的数据文件和划分的目标路数 K，输出一份后缀为 “.hgr.part.K” 的数据文件。

HGR (.hgr) 文件内容为：第一行有四个整数，第一个整数为 0 或 1（表示下列数据的起始索引），第二、三个整数分别超图的顶点数（vertices，对应原始数据文件的 node 数）和网格数（nets，对应原始数据文件的 net 数），第四个整数为总指针数（pins，对应原始数据文件各个 net 的 node 数的总和）；从第二行开始，每一行表示同一个 net 中的 vertice 编号（对应原始数据同一个 net 中的 node 编号），不同行表示不同的 net（对应原始数据同理）。以用图 1 的数据模拟得到图 6 的(a)为例。

输出文件的内容为顶点数（vertices）个 0 到 K-1 之间的整数，每一行只有一个整数，其表示以相应行为编号（以起始索引为准）的顶点被划分在那一条路上。以用图 6 的(a)为输入进行 2 路划分的输出(b)为例。

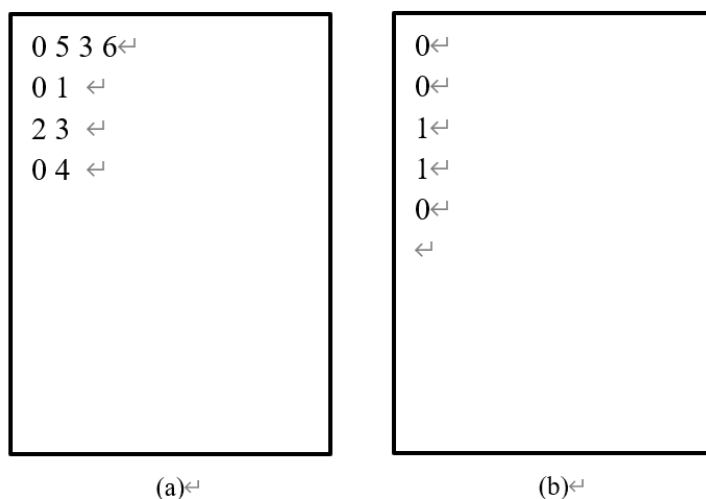


图 6 (a) HGR 文件示例，如图中第一行各整数分别表示以 0 为起始索引，5 个顶点，3 个网格，共计 6 个引脚；第二行开始，第一个网格包含顶点 0 和 1，第二个网格包含顶点 2 和 3，第三个网格包含顶点 0 和 4。(b) PaToH 输出文件示例，文件后缀为 “.hgr.part.2”，如图中顶点 0、1 和 4 被划分到路 0；顶点 2 和 3 被划分到路 1。

## 2.4 优化算法

### 2.4.1 约束条件（划分阈值）

受本人在课题研究的现实约束，本课题的约束条件（划分阈值）有两个：不同芯

片内部资源利用率的平均差异百分比与各个芯片间的物理连线条数。

不同芯片内部资源利用率的平均差异：由于各个芯片内部的资源是有限、相同且等量的，记该指标为  $a$ ，则  $a$  的计算公式如下，

$$a = \frac{C_{max} - C_{min}}{C_{min}}$$

其中， $C_{max}$  和  $C_{min}$  分别为所有芯片中划分到最多 node 的 die 中的 node 数和划分到最少 node 的 die 中的 node 数。

各个芯片间物理连线：参考文献<sup>[5][9]</sup>，该指标的计算方法因多芯片 FPGA 电路布局设计者对数据流的不同设计而不同。通常有以下数据流设计，

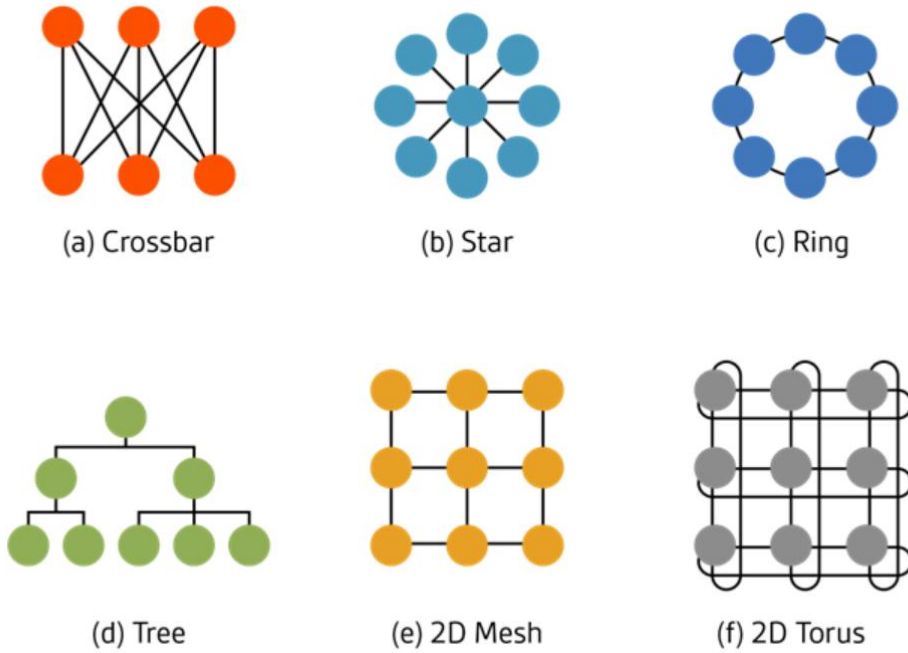


图 7 各种数据流设计

1) “星形” (Star) 数据流<sup>[18]</sup>：如图 7 中的(b)所示，由一个 node 向其它 node 传递数据，故数据统一从一个 die 传递到另一个 die，则该设计下的物理连线条数  $l$  的计算方法如下，

$$l = \sum_i node_{net_i} - 1$$

其中， $node_{net_i}$  为第  $i$  个 net 中的 node 的个数。

2) “链式” (Chain) 数据流<sup>[18]</sup>：类似图 7 中的(c)所示，net 中各个 node 所在的 die 以链条的形式“串起来”，数据在链条上一个 die 一个 die 的传递，则该设计下的物理连线条数  $l$  的计算方法如下，

$$l = \sum_i Die_{net_i} - die_{net_i}$$

其中,  $Die_{net_i}$  和  $die_{net_i}$  分别为与 net 相交的所有 die (net 中部分 node 被划分到的 die) 中索引的最大值和最小值。

3) “网格” (Mesh) 数据流<sup>[18]</sup>: 如图 7 中的(e)所示, 数据在 die 间的传递如同网格一般, 则该设计下的物理连线条数较难计算。

本课题基于以上两个约束条件, 以贪婪放置 (Greedy Placement) 为基础对优化算法进行设计。

### 2.4.2 贪婪放置 (Greedy Placement)

假设我们得到的初始划分结果的不同芯片内部资源利用率的平均差异大于阈值 (由于 PaToH 划分器<sup>[17]</sup>的划分效果, 我们的初始划分方案为平衡划分, 故不同芯片内部资源的平均差异大部分情况下不会超过阈值), 以贪婪放置为设计基础, 我们可以把划分到更多 node 的 die 拿出部分移到划分到更少 node 的 die 中。具体设计详见 2.4.3。

假设我们得到的初始划分结果的各个芯片间物理连线大于阈值, 以贪婪放置为设计基础, 我们可以找到所需物理连线最多的 net, 将与其相交的所有 die 中最远的 die 中的 node 移到近一点的 die 上。具体设计详见 2.4.4。

不难发现, 在上述假设中, 只要我们在基于不同芯片内部资源利用率的平均差异进行优化时合理地将部分 node 换到新的 die 中, 得到的划分方案的各个芯片间的物理连线条数可以不会发生改变, 但当我们在基于各个芯片间的物理连线条数进行优化时, 我们无法保证得到的划分方案的不同芯片内部资源利用率的平均差异保持不变。

因此, 基于上述推理研究所得, 在优化优先级上我们应先基于各个芯片间的物理连线进行优化, 再基于不同芯片内部资源利用率的平均差异进行优化 (即实际操作时, 先进行 2.4.4 中的操作, 再进行 2.4.3 中的操作)。

### 2.4.3 基于不同芯片内部资源利用率平均差异的阈值的优化

顺着 2.4.2 中基于不同芯片内部资源利用率的平均差异的优化方法的思路, 我们可以设计出如下优化算法,

---

输入: NETS 文件, 一个 node 类的数组对象 node\_list, 一个 die 类的数组对象 die\_list。

输出: 新的 node\_list, 新的 die\_list。

---

//a)设置参数

d\_sorted[], net\_in\_die[], sum\_net\_in\_die = 0, net\_C\_M = 0, i\_net\_C\_M = -1;

初始化参数;

以 die\_list.capacity 为基准从大到小排序, 并将对应 die 的索引按顺序存入 d\_sorted;

//b)寻找拥有最多 node 的 net

一行一行读取 NETS 文件:

空行跳过;

每行第一个字符串为“net”: 跳过;

每行第一个字符串为“endnet”:

若 net\_in\_die[d\_sorted[0]] = 1:

若此时的 node 的索引指针大于 net\_C\_M:

则 net\_C\_M 为此时的 node 的索引指针的值且 i\_net\_C\_M 为此  
时 net 的索引;

重置 net\_in\_die, sum\_net\_in\_die 和 node 的索引指针后跳过;

记本行 node 在 node\_list 的索引为 r;

net\_in\_die[node\_list[r].die] = 1;

node 的索引指针自增;

//c)变更 node 所在的 die

重置 net\_in\_die;

mode = -1;

一行一行读取 NETS 文件:

空行跳过;

每行第一个字符串为“net”: 跳过;

每行第一个字符串为“endnet”:

若此时 net 的索引为 i\_net\_C\_M:

sum\_net\_in\_die 为 net\_in\_die 中所有数值的和;

若 sum\_net\_in\_die 不为 1: 则 mode = 0;

否则: mode = 1;

退出文件;

否则: 跳过;

若此时 net 的索引不是 i\_net\_C\_M: 则跳过;

若此时 net 的索引是 i\_net\_C\_M:

记本行 node 在 node\_list 的索引为 r;

net\_in\_die[node\_list[r].die] = 1;



---

一行一行读取 NETS 文件:

空行跳过;

每行第一个字符串为 “net” : 跳过;

每行第一个字符串为 “endnet” :

若此时 net 的索引为  $i\_net\_C\_M$ : 退出文件;

否则: 跳过;

若此时 net 的索引不是  $i\_net\_C\_M$ : 则跳过;

若此时 net 的索引是  $i\_net\_C\_M$ :

若  $mode = 1$ :

将本行的 node 转移到第  $d\_sorted[MAX]$  个 die;

若  $mode = 0$ :

若本行的 node 所在 die 为  $d\_sorted[0]$ : 则将这个 node 优先转移到第  $d\_sorted$  的索引越大的数值个 die 中, 且该 die 所对应的  $net\_in\_die$  为 1;

//d) 更新 die 中的相关信息

更新  $die\_list$ ;

---

步骤 a) 中, 设置了  $d\_sorted$ ,  $net\_in\_die$ ,  $sum\_net\_in\_die$ ,  $net\_C\_M$  以及  $i\_net\_C\_M$  四个参数 (数组), 分别用来存放 die 所有的 node 数的从大到小排序后的索引顺序, net 与 die 相交与否 (相交为 1, 否则为 0),  $net\_in\_die$  中所有数值的和 (表示 net 与多少个 die 相交, 用于确定如何对 node 进行移动操作, 详见步骤 c)), 所有 net 中拥有最多 node 的 net 的 node 所有数以及其对应的索引。

步骤 b) 中, 读取 NETS 文件来获得各个 net 的 node 信息, 再根据  $node\_list$  的数据信息找到所有 net 中拥有最多 node 的 net 的 node 所有数 ( $net\_C\_M$ ) 以及其对应的 net 的索引 ( $i\_net\_C\_M$ )。

步骤 c) 中, 第一遍读取 NETS 文件来确定与所有 net 中拥有最多 node 的 net 相交的 die 的总数  $sum\_net\_in\_die$ , 若该值为 1, 则使用方案 2 进行 node 转移 (即将 node 从旧的 die 移动到新的 die 中), 否则使用方案 1。在第二遍读取 NETS 文件时, 选择不同方案进行操作:

方案 1, 将在第  $d\_sorted[0]$  个 die 上的 node 优先移动到对应  $d\_sorted$  索引越大且与本 net 相交的 die 中; (这样操作可以确保各个芯片之间的物理连线条数保持不变)

方案 2, 将 net 中所有 node 移动到  $d\_sorted$  索引最大的数值所对应的 die 中。(有我们所设计出的优化算法可推理出该 net 中所有的 node 均在第  $d\_sorted[0]$  个 die 中, 本方案相当于把位于拥有最多 node 的 die 中的 net 整个移动到拥有最少 node 的 die 中)

## 2.4.4 基于各个芯片间物理连线的阈值的优化（以“链式”数据流设计为例）

顺着2.4.2 中基于各个芯片间的物理连线的优化方法的思路，我们可以设计出如下优化算法，

---

输入：NETS 文件，一个 node 类的数组对象 node\_list，一个 die 类的数组对象 die\_list。

输出：新的 node\_list，新的 die\_list。

---

//a)设置参数

d\_M = 0, d\_m = 3, net\_SLL\_M = 0, i\_net\_SLL\_M = -1;

//b)寻找所需物理连线最多的 net

一行一行读取 NETS 文件：

空行跳过；

每行第一个字符串为“net”：跳过；

每行第一个字符串为“endnet”：

若  $d_M - d_m > \text{net\_SLL\_M}$ ：

则  $\text{net\_SLL\_M} = d_M - d_m$  且  $i\_net\_SLL\_M$  为此时 net 的索引；

重置 d\_M 和 d\_m 后跳过；

若本行的 node 所在的 die 更远：则更新 d\_M；

若本行的 node 所在的 die 更近：则更新 d\_m；

//c)变更 node 所在 die

重置 d\_M 和 d\_m；

一行一行读取 NETS 文件：

空行跳过；

每行第一个字符串为“net”：跳过；

每行第一个字符串为“endnet”：跳过；

此时 net 的索引不是  $i\_net\_SLL\_M$ ：跳过；

此时 net 的索引是  $i\_net\_SLL\_M$ ：

若本行的 node 所在的 die 更远：则更新 d\_M；

若本行的 node 所在的 die 更近：则更新 d\_m；

一行一行读取 NETS 文件：

空行跳过；

每行第一个字符串为“net”：跳过；

---

---

每行第一个字符串为“endnet”：跳过；

此时 net 的索引不是 i\_net\_SLL\_M：跳过；

此时 net 的索引是 i\_net\_SLL\_M：

若本行的 node 所在 die 为 d\_M：则将这个 node 转移到第 d\_M - 1 个 die 中；

（或者，若本行的 node 所在 die 为 d\_m：则将这个 node 转移到第 d\_m + 1 个 die 中；）

//d)更新 die 中的相关信息

更新 die\_list;

---

步骤 a)中，设置了 d\_M, d\_m, net\_SLL\_M 以及 i\_net\_SLL\_M 四个参数，分别用来存放 net 中的 node 所在的 die 的索引的最大值，net 中的 node 所在的 die 的索引的最小值，所有 net 中所需物理连线最多的 net 的所需物理连线条数以及其对应的 net 的索引。

步骤 b)中，读取 NETS 文件来获得各个 net 的 node 信息，再根据 node\_list 的数据信息找到所有 net 中所需物理连线最多的 net 的所需物理连线条数（net\_SLL\_M）以及其对应的 net 的索引（i\_net\_SLL\_M）。

步骤 c)中，第一遍读取 NETS 文件来确定所有 net 中所需物理连线最多的 net 中的 node 所在的 die 的索引的最大值（d\_M）和最小值（d\_m），有助于在第二次读取 NETS 文件时对该 net 中的部分 node 进行转移操作（即将 node 从旧的 die 移动到新的 die 中），要么把最远的 node 移近（即 node\_list[.die 从 d\_M 变为 d\_M - 1），要么把最近的 node 移远（即 node\_list[.die 从 d\_m 变为 d\_m + 1）。

### 第三章 算法实践检验

以附件3中的 clk\_design1 为例进行实践检验。

实现本实践的划分算法的 C++源代码详见附件2。

本实践是将 clk\_design1 中的 node 划分到 Xilinx VU13P (4-dies)中。该多芯片 FPGA 电路有四个芯片，并且是  $1 \times 4$  排列（如图 8 所示），因此我选择使用“链式”数据流设计进行物理布局，这意味着在计算物理连线时，若节点甲和节点乙相连且甲在第一个芯片而乙在第四个芯片，则需要 3 条物理连线才能将两个节点进行相连，依次类推。

本实践约束条件（阈值）为不同芯片内部资源利用率的平均差异 $\leq \pm 20\%$ ，且各个 die 之间的物理连线数（采用链式数据流策略）要求 $\leq 650$  条。

具体操作流程详见图 9。

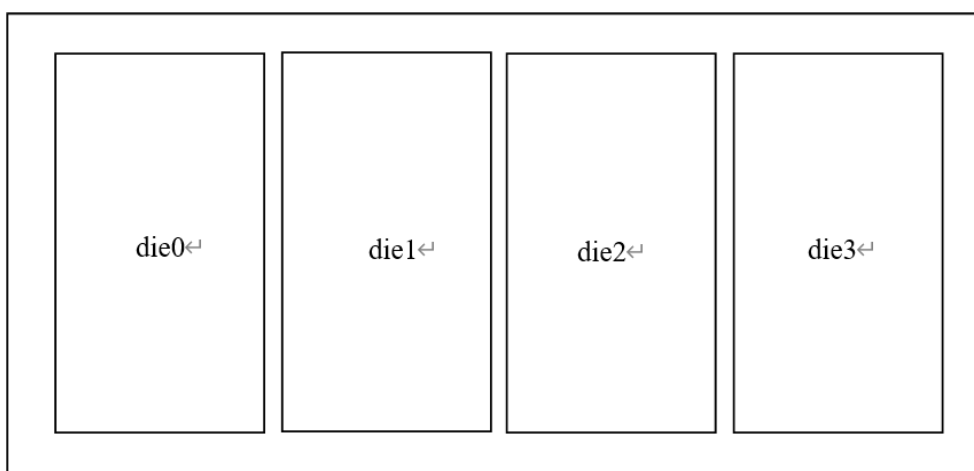


图 8

```

D:\毕设\毕设附件\2.算法cpp代码\cpp_code\data\clk_design1\design
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
0
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
2
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
3
The average cell using is: 0.00364964
The SLL using is: 660
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
4
Please input the requested average cell using and SLL:
0.2 650
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
3
The average cell using is: 0.0405126
The SLL using is: 784
Please select the mode:
0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for optimization, others for exit.
5

D:\毕设\毕设附件\2.算法cpp代码\cpp_code\x64\Debug\毕设.exe (进程 67456)已退出, 代码为 0 (0x0).
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .

```

图 9 代码运行具体过程, C++源代码详见附件2, 其中, 操作 0 为前期准备、操作 1 是初始划分、操作 2 是完善数据、操作 3 为计算指标、操作 4 为优化, 操作 5 为退出程序。

注: 由于 PaToH 划分器为闭源项目, 故本次实践检验前已完成初始划分操作, 源代码文件仅拥有 PaToH 划分器的资源压缩包, 文件名为.../2.算法 cpp 代码/cpp\_code/patoh-Linux-x86\_64.tar.gz, 文件来源为 <https://cc.gatech.edu/~umit/software.html>。

### 3.1. 数据数字化

对应图 9 中从开始到输入 0 后再次弹出提示的过程。

输入文件名称 (不加后缀), 即输入 .../clk\_design1/design。

成功输出对应 HGR 文件, 详见附件3中的 clk\_design1/design.hgr 文件。

经确认检验, 所生产的 HGR 文件符合初始划分的使用标准, 且可知 clk\_design1 数据中有 9882 个 node、9892 个 net 以及 13 种 cell。

### 3.2. 初始划分与约束相关参数

因某些因素未与图 9 任何操作对应, 详见图 4 的注解。

输入文件名称 (不加后缀) 与划分路数, 即输入 .../clk\_design1/design, 4。

成功输出对应文件, 详见附件3中的 clk\_design1/design.hgr.part.4 文件。

经检验, 上述文件即为一种平衡划分方案, 其中各个 die 中划分到的 node 分别有 2475 个、2466 个、2471 个、2470 个。各个 die 的资源利用情况及资源利用率如表 2 和表 3 所示。

计算同芯片内部资源利用率的平均差异百分比以及各个 die 之间的物理连线条数

（详见图 10）可知，初始划分后，不同芯片内部资源利用率的平均差异约为 0.365%，各个 die 之间的物理连线有 660 条。

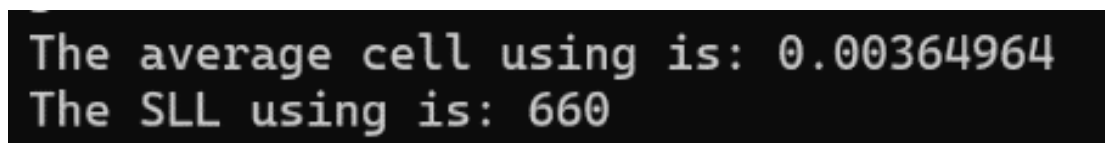


图 10 初始划分后的芯片内部资源利用率的平均差异与各个芯片间的物理连线条数

| die 编号   | LUT    | FF     | CARRY8 | DSP48E2 | RAMB36E2 | IO  |
|----------|--------|--------|--------|---------|----------|-----|
| 1        | 1245   | 1210   | 0      | 1       | 0        | 19  |
| 2        | 1240   | 1208   | 0      | 0       | 1        | 17  |
| 3        | 1229   | 1218   | 0      | 1       | 1        | 22  |
| 4        | 1209   | 1210   | 0      | 0       | 0        | 51  |
| die 可用资源 | 432000 | 864030 | 54000  | 3072    | 672      | 208 |

表 2 各个 die 的资源利用情况

| die 编号 | LUT (%) | FF (%) | CARRY8 (%) | DSP48E2 (%) | RAMB36E2 (%) | IO (%) |
|--------|---------|--------|------------|-------------|--------------|--------|
| 1      | 0.288   | 0.140  | 0          | 0.033       | 0            | 9.135  |
| 2      | 0.287   | 0.140  | 0          | 0           | 0.149        | 8.173  |
| 3      | 0.285   | 0.140  | 0          | 0.033       | 0.149        | 10.58  |
| 4      | 0.280   | 0.140  | 0          | 0           | 0            | 24.52  |

表 3 各个 die 的资源利用率

### 3.3. 优化与结果

对应图 9 中输入 4 后所有的操作。

输入阈值，即输入 0.2，650。

成功输出对应文件，详见附件 3 中的 clk\_design1/design.hgr.part.4.optimized 文件。

进行计算操作后，发现不同芯片内部资源利用率的平均差异小于阈值，为 4.05%，但各个 die 之间的物理连线条数仍然超过阈值，为 784 条（详见图 11）。

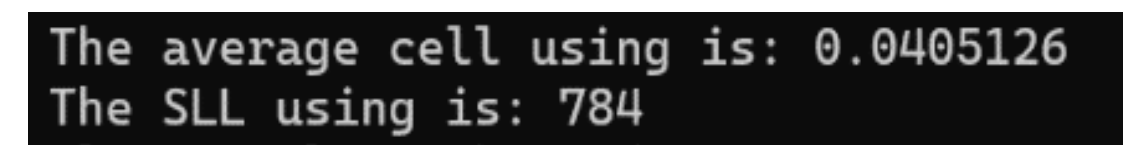


图 11 优化后的芯片内部资源利用率的平均差异与各个芯片间的物理连线条数

| die 编号   | LUT    | FF     | CARRY8 | DSP48E2 | RAMB36E2 | IO  |
|----------|--------|--------|--------|---------|----------|-----|
| 1        | 1245   | 1210   | 0      | 1       | 0        | 19  |
| 2        | 1240   | 1259   | 0      | 0       | 1        | 17  |
| 3        | 1229   | 1218   | 0      | 1       | 1        | 22  |
| 4        | 1209   | 1159   | 0      | 0       | 0        | 51  |
| die 可用资源 | 432000 | 864030 | 54000  | 3072    | 672      | 208 |

**表 4** 优化后各个 die 的资源利用情况。与表 2 相比，优化后 die2 和 die4 对资源 FF 的利用情况发生改变，die2 对资源 FF 的利用量从 1208 增加至 1259，die4 对资源 FF 的利用量从 1210 下降至 1159。

| die 编号 | LUT<br>(%) | FF<br>(%) | CARRY8<br>(%) | DSP48E2<br>(%) | RAMB36E2<br>(%) | IO<br>(%) |
|--------|------------|-----------|---------------|----------------|-----------------|-----------|
| 1      | 0.288      | 0.140     | 0             | 0.033          | 0               | 9.135     |
| 2      | 0.287      | 0.146     | 0             | 0              | 0.149           | 8.173     |
| 3      | 0.285      | 0.140     | 0             | 0.033          | 0.149           | 10.58     |
| 4      | 0.280      | 0.134     | 0             | 0              | 0               | 24.52     |

**表 5** 优化后各个 die 的资源利用率。同理，与表 3 相比，die2 对资源 FF 的利用率从 0.140% 增加至 0.146%，die4 对资源 FF 的利用率从 0.140% 下降至 0.134%。

对原始数据文件进行分析研究发现：同一个 node 可以属于多个 net。这个发现可以推出：在优化过程中，对某一个 net 的 node 进行转移操作时，虽然该 net 的物理连线变少了，但其它 net 的物理连线可能增加了，若增加到更多，就会导致优化后的划分方案的各个 die 之间的物理连线反而增多的情况。

因此，我选择 clk\_design1/design.hgr.part.4 文件作为最优的划分方案。

## 第四章 总结

### 4.1. 课题总结

通过本课题的研究，我了解了多芯片 FPGA 电路的划分的意义与原理，还包括作为多芯片 FPGA 电路的划分算法的理论支撑的（超）图划分，并在国内外前辈们的研究成果的基础上，完成了将原始数据中的 `clk_design1` 相关数据划分到 Xilinx VU13P (4-dies) 的四个芯片中的任务，将划分结果的不同芯片内部资源利用率的平均差异控制在 0.365% 左右，各个 die 之间的物理连线控制在 660 条，一定程度上证明了这套多芯片 FPGA 电路的划分算法在万门级情况下的可行性和有效性。

不过，本课题还存在不少可以改进的地方，例如，

- 1) 实践检验是，初始划分结果较好，致使算法中的优化部分的有效性的可信度不高。
- 2) 算法设计时是分步设计，并未将整个划分流程合并。
- 3) 受硬件影响，该算法还未在 `node` 数超过 10000 的数据，即引脚数不超过万门级的多芯片 FPGA 电路的划分任务中实践检验。

相应的研究思路为，

- 1) 两种思路：一种选择比 PaToH 划分器较弱的划分器进行初始划分操作；一种选择其它数据使得 PaToH 划分器的划分结果并非最优。
- 2) 在验明算法的可行性和有效性后，将算法合并，并用递归策略完成多次优化操作。
- 3) 稍微修改 C++ 代码后，在存储空间更大的计算机上对 `node` 数更多的数据进行划分任务。

### 4.2. 未来展望

FPGA 电路正从可编程逻辑器件转变为异构融合、动态自适应的通用算力基座。在架构层面，多晶粒集成和三维堆叠将单器件逻辑容量增加到数亿门级，同时高速芯片间的互连和统一编程模型将多芯片 FPGA 阵列虚拟为单一巨型加速器。动态部分可重构能力将得到提升，通过芯片上集成低延迟重构控制器和智能划分处理器，FPGA 能够在短时间内按需重组成不同的硬件管线，实时响应负载变化。设计工具方面人工智能驱动的自动优化将贯穿综合、划分、布局和布线全流程，图划分技术被深度嵌入编译栈，自主搜索算子切分、数据流架构与物理资源的最优映射，使开发门槛大幅降



低。面向数据中心、自动驾驶和 AI Infra 等领域，FPGA 将作为智能网卡、近存储计算和领域特定加速器的核心，与 GPU、CPU 构成深度耦合的计算单元，为自适应和高能效计算持续开拓边界。

作为基于划分的 FPGA 布局技术的理论基础，图划分技术正从静态、单目标优化向智能化、动态化和极限规模处理全面推进。未来，基于图神经网络和强化学习的划分算法将能够自主学习电路网表、社交网络等超大规模图的结构特征，在划分初期就预判关键路径和拥塞热点，生成最优解。面对数亿边的图，分布式流式划分框架将允许在不加载全图的情况下进行在线划分，支撑动态图数据的实时处理。三维集成与 Chiplet 时代的到来，将平面图划分问题推向多层三维空间的图划分问题。此外，图划分将与存算一体、近存计算等新架构深度融合，在划分阶段就协同规划数据放置与计算调度，突破“存储墙”瓶颈。图划分将不再只是离线预处理，而是贯穿数据流处理、芯片设计与科学计算全生命周期的核心技术。

正如1.1中所言，对与 FPGA 相关的研发、设计技术的需求与日俱增。未来十年，FPGA 行业将在高速增长中实现价值跃迁。随着架构持续融合、工艺制程及工具链逐步突破，FPGA 将深度融入 AI 与通信技术的核心脉络；同时，其独特定位与构建生态的强大能力，也将为整个智能计算领域注入更坚实的创新动能。而作为与 FPGA 相关的研发、设计技术的一环，多芯片 FPGA 电路的划分技术也会在未来不断发展，朝着更智能、更自动化、更系统级的优化的方向不断更新迭代。

## 附件

1. 原始数据文件实例
2. C++程序
3. 实践实例

## 参考文献

- [1] 田春生,陈雷,王源,等.面向 FPGA 的布局与布线技术研究综述[J].电子学报,2022,50(05):1243-1254.
- [2] 黄宏赞,王子健,惠锋,等.一种多维约束感知的多裸片 FPGA 网表划分方法[J/OL].微电子学与计算机,1-6[2026-04-15].<https://link.cnki.net/urlid/61.1123.tn.20260319.1313.016>.
- [3] TRIMBERGER S M. Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology[J]. Proceedings of the IEEE, 2015, 103(3): 318-331.
- [4] TRIMBERGER S M. Three ages of FPGAs: a retrospective on the first thirty years of FPGA technology: this paper reflects on how Moore's law has driven the design of FPGAs through three epochs: the age of invention, the age of expansion, and the age of accumulation[J]. IEEE Solid-State Circuits Magazine, 2018, 10(2): 16-29.
- [5] KHATKHATE A, LI C, AGNIHOTRI A R, et al. Recursive bisection based mixed block placement[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2005, 24(5): 748-761.
- [6] BUSTANY I, GASPRAYAN G, KAHNG A B, et al. An open-source constraints-driven general partitioning multi-tool for VLSI physical design [C] // IEEE/ACM International Conference on Computer-Aided Design (ICCAD). San Francisco, CA, USA. 2023: 1-9. DOI: 10.1109/ICCAD57390.2023.10323975.
- [7] ÇATALYÜREK Ü, DEVINE K, FARAJ M, et al. More recent advances in (hyper)graph partitioning [J]. ACM Computing Surveys, 2023, 55(12): 1-38. DOI: 10.1145/3571808.
- [8] 路文军,刘红卫,曹欢欢.改进的宽度优先超图划分生成方法[J].哈尔滨师范大学自然科学学报,2025,41(03):5-10.
- [9] 李本正.多 FPGA 系统结构感知的电路快速划分方法研究[D].西安电子科技大学,2023.DOI:10.27389/d.cnki.gxadu.2023.004275.
- [10] 马永刚.图划分方法及其在分布式网络环境下的应用[D].大连理工大学,2012.
- [11] 尚俊霖,张振宇,屈稳稳,等.面向分布式图计算的图划分技术综述[J].计算机研究与发展,2025,62(01):90-103.
- [12] 李金吉,张岩峰,巩树凤,等.流式处理的异步图处理框架[J].软件学报,2018,29(03):528-544.DOI:10.13328/j.cnki.jos.005441.
- [13] 李贺,刘延娜,袁航,等.动态图划分算法研究综述[J].软件学报,2023,34(02):539-564.DOI:10.13328/j.cnki.jos.006705.
- [14] Fischer E, Matsliah A, Shapira A. Approximate hypergraph partitioning and applications

- [J].SIAM Journal on Computing, 2020, 36(7):3155-3185.
- [15] Buluç A, Meyerhenke H, Safro I et al. Recent advances of graph partitioning[M]. Berlin: Springer International Publishing, 2016.
- [16] Demir E, Aykanat C, Cambazoglu B B. Clustering spatial networks for aggregate query processing: A hypergraph approach[J]. Information Systems, 2008, 33(1): 1-17.
- [17] Catalyurek U, Aykanat C. PaToH: Partitioning Tool for Hypergraphs (Manual)[CP/OL]. [2026-05-07]. <https://faculty.cc.gatech.edu/~umit/PaToH/manual.pdf>.
- [18] NIGHTINGALE A. Network-on-chip interconnect topologies explained[EB/OL]. (2023-08-03)[2026-05-07]. <https://www.ednasia.com/network-on-chip-interconnect-topologies-explained/>
- [19] IMANI M, ZOU Z W, BOSCH S, et al. Revisiting hyperdimensional learning for FPGA and low-power architectures[C]. 2021 IEEE International Symposium on High-Performance Computer Architecture(HPCA). South Korea: IEEE, 2021: 221-234.
- [20] ZHANG J X, YANG T, LI Q Z, et al. An FPGA-based neural network overlay for ADAS supporting multi-model and multi-mode[C]. 2021 IEEE International Symposium on Circuits and Systems(ISCAS). South Korea: IEEE, 2021: 1-5.
- [21] KAN H W, LI R G, SU D D, et al. Trusted edge cloud computing mechanism based on FPGA cluster[C]. 2020 IEEE 8th International Conference on Computer Science and Network Technology(ICCST). China: IEEE, 2020. 146-149.
- [22] 刘公绪, 史凌峰, 辛东金 . 基于 FPGA 的零误差大数阶乘算法的设计与实现[J]. 电子学报, 2019, 47(5): 1180-1184.
- [23] 蹇强, 张培勇, 王雪洁 . 一种可配置的 CNN 协加速器的 FPGA 实现方法[J]. 电子学报, 2019, 47(7): 1525-1531.
- [24] SADEGHI A, LIGHVAN M Z, PRINETTO P. Automatic and simultaneous floorplanning and placement in field-programmable gate arrays with dynamic partial reconfiguration based on genetic algorithm[J]. Canadian Journal of Electrical and Computer Engineering, 2020, 43(4): 224-234 .
- [25] HERATH K, PRAKASH A, JIANG G Y, et al. Ant Colony Optimization based module footprint selection and placement for lowering power in large FPGA designs[C]. 2018 International Conference on ReConFigurable Computing and FPGAs (ReConFig). Mexico: IEEE, 2018: 1-8.
- [26] SARI A, PSARAKIS M. Scrubbing-aware placement for reliable FPGA systems[J]. IEEE Transactions on Emerging Topics in Computing, 2020, 8(3): 564-576.
- [27] STEINBRUNN M, MOERKOTTE G, KEMPER A. Heuristic and randomized

- optimization for the join ordering problem[J]. The VLDB Journal, 1997, 6: 191-208.
- [28] CARL S. VLSI Placement and Global Routing Using Simulated Annealing[M]. Boston, MA, USA: Springer Science & Business Media, 2012.
- [29] GOPALAKRISHNAN P, LI X, PILEGGI L. Architecture-aware FPGA placement using metric embedding[C]. 2006 43<sup>rd</sup> ACM/IEEE Design Automation Conference. USA: IEEE, 2006: 460-465.
- [30] LIN T H, BANERJEE P, CHANG Y W. An efficient and effective analytical place for FPGAs[C]. 2013 50<sup>th</sup> ACM/EDAC/IEEE Design Automation Conference (DAC). USA: IEEE, 2014: 1-6.
- [31] KIM, M C, LEE D J, MARKOV I L. SimPL: an effective placement algorithm[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2012, 31(1): 50-60.

## 附录 1

C++源代码如下:

```
class cell
{
public:
    std::string name;//the name of the cell
    std::string output_pin[360];//the name of the output pin of the cell
};

class node
{
public:
    std::string name;//the name of the node
    int cell;//the index of the cell
    int net;//the index of the net
    int die;//the index of the die
};

class net
{
public:
    std::string name;//the name of the net
};

class die
{
public:
    int capacity = 0;
    int list[5000];//list of nodes' index
    void add_to_die(int a);//add node to die
    void del_from_die(int a);//delet node from die
};

void die::add_to_die(int a)
{
    capacity++;
    if (capacity > 5000)
    {
        std::cout << "Die's capacity is overflow!" << std::endl;
    }
    else
    {
        list[capacity - 1] = a;
    }
}
```

```

}
void die::del_from_die(int a)
{
    int j;//a's index
    for (int i = 0; i < capacity; i++)
    {
        if (list[i] == a)
        {
            j = i;
            break;
        }
    }
    for (int i = j; i < capacity; i++)
    {
        list[i] = list[i + 1];
    }
    capacity--;
}

void init(string file_name, die *die_list, node *node_list, cell *cell_list, int
&n_cells, int &n_nets, int &n_pins, int &n_nodes)
//input: the name of file;
//output: 4 dies, 10000 nodes, the number of cells, nets and pins in the file, the net
list
{
    int i_cell = -1, j_opin = 0;
    int i_pin = 0;
    int i_net = -1;
    n_cells = 0;
    n_nets = 0;
    n_pins = 0;
    n_nodes = 0;
    //initialize or clear the node list
    for (int i = 0; i < 10000; i++)
    {
        node_list[i].name = "";
        node_list[i].cell = -1;
        node_list[i].net = -1;
        node_list[i].die = -1;
    }
    //initialize cell list this research needs
    cell_list[0].name = "LUT1";
    cell_list[1].name = "LUT2";
    cell_list[2].name = "LUT3";
    cell_list[3].name = "LUT4";

```

```
cell_list[4].name = "LUT5";
cell_list[5].name = "LUT6";
cell_list[6].name = "FDRE";
cell_list[7].name = "CARRY8";
cell_list[8].name = "DSP48E2";
cell_list[9].name = "RAMB36E2";
cell_list[10].name = "IBUF";
cell_list[11].name = "OBUF";
cell_list[12].name = "BUFGCE";
n_cells = 13;
//open the files
ifstream file1(file_name + ".lib");
ifstream file2(file_name + ".nets");
ifstream file3(file_name + ".nodes");
ifstream file4(file_name + ".scl");
//check if the files are opened successfully
if(!file1.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
    return;
}
else if(!file2.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
    return;
}
else if(!file3.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
    return;
}
else if(!file4.is_open())
{
    cout << "Cannot open file: " << file_name << endl;
    return;
}

string line;
//complete the cell list based on the .lib file
while (getline(file1, line))
{
    if (line.empty()) continue;

    istringstream iss(line);
```



```

string token1, token2, token3;
if (!(iss >> token1)) continue;
if (!(iss >> token2)) continue;
if (!(iss >> token3)) continue;

if (token1 == "CELL")
{
    for (int r = 0; r < n_cells; r++)
    {
        if (cell_list[r].name == token2)
        {
            i_cell = r;
            break;
        }
    }
    continue;
}
if (token1 == "END")
{
    i_cell = -1;
    j_opin = 0;
    continue;
}

if (token3 == "OUTPUT" && i_cell != -1)
{
    cell_list[i_cell].output_pin[j_opin] = token2;
    j_opin++;
}
}
file1.close();
file4.close();
//initialize the node list and the cell list based on the .nodes file
while (getline(file3, line))
{
    if (line.empty()) continue;

    istringstream iss(line);
    string token1, token2;
    if (!(iss >> token1)) continue;
    if (!(iss >> token2)) continue;
    //write down the name of the node
    node_list[i_pin].name = token1;
    //find and write down the cell index of the node

```

```
int found_cell = -1;
if (!token2.empty())
{
    for (int r = 0; r < n_cells; ++r)
    {
        if (cell_list[r].name == token2)
        {
            found_cell = r;
            break;
        }
    }
}
node_list[i_pin].cell = found_cell;
n_nodes++;

++i_pin;
if (i_pin > 10000) break;
}
file3.close();
//initialize the node list and based on the .nets file
while (getline(file2, line))
{
    if (line.empty()) continue;

    istringstream iss(line);
    string token1, token2;
    if (!(iss >> token1)) continue;
    if (!(iss >> token2)) continue;

    if (line == "endnet") continue;
    if (token1 == "net")
    { i_net++; n_nets++; continue; }
    //write down the net index of the node
    n_pins++;
    int found_pin = -1;
    for (int r = 0; r < n_pins; ++r)
    {
        if (node_list[r].name == token1)
        {
            found_pin = r;
            break;
        }
    }
}
```

```

        node_list[found_pin].net = i_net;

        if (i_net > 10000) break;
    }
    file2.close();
}

void create_hgr(string file_name, node *node_list, int n_nodes, int n_nets, int n_pins)
//create a .hgr file for partitioning
{
    ofstream file;
    file.open(file_name + ".hgr", ios::out);
    ifstream File(file_name + ".nets");

    if (!File.is_open())
    {
        cout << "Cannot open nodes file: " << file_name + ".nets" << endl;
        return;
    }

    file << "0" << " " << n_nodes << " " << n_nets << " " << n_pins << endl;

    string line;
    while (getline(File, line))
    {
        if (line.empty()) continue;

        istringstream iss(line);
        string token1;
        if (!(iss >> token1)) continue;

        if (token1 == "net") continue;
        if (line == "endnet") { file << endl; continue; }

        int found_pin = -1;
        for (int r = 0; r < n_pins; ++r)
        {
            if (node_list[r].name == token1)
            {
                found_pin = r;
                break;
            }
        }
        file << found_pin << " ";
    }
}

```

```
File.close();
file.close();
}

void place(string f_name, int n_dies, node *node_list)
{
    //clear the die information in the node list
    for (int r = 0; r < 10000; r++)
    {
        node_list[r].die = -1;
    }

    ifstream file(f_name + ".hgr.part." + to_string(n_dies));
    if (!file.is_open())
    {
        cout << "Cannot open nodes file: " << f_name + ".hgr.part." +
to_string(n_dies) << endl;
        return;
    }
    string line;
    int i_node = 0;
    while (getline(file, line))
    {
        if (line.empty()) continue;

        istringstream iss(line);
        string d;
        while (iss >> d)
        {
            node_list[i_node].die = stoi(d);
            i_node++;
        }
    }
    file.close();
}

void check(string file_name, cell *cell_list, node *node_list, die *die_list, int
&bridge, float &average_cell_using)
//caculate the number of bridges and the average cell using of the partitioning result
{
    bridge = 0;

    ifstream file1(file_name + ".nets");

    if (!file1.is_open())
```

```

{
    cout << "Cannot open file: " << file_name + ".nets" << endl;
    return;
}
//caculate the average cell using
int c_M = 0;
int c_m = 5000;
for (int i = 0; i < 4; i++)
{
    if (die_list[i].capacity > c_M) c_M = die_list[i].capacity;
    if (die_list[i].capacity < c_m) c_m = die_list[i].capacity;
}
average_cell_using = (c_M - c_m) * 1.0 / c_m;
//caculate the SLL using
int d_M = 0, d_m = 3;
string line;
while (getline(file1, line))
{
    if (line.empty()) continue;

    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;

    if (token == "net") continue;
    if (token == "endnet")
    {
        bridge += (d_M - d_m);
        d_M = 0;
        d_m = 3;
        continue;
    }

    int found_node = -1;
    for (int r = 0; r < 10000; r++)
    {
        if (node_list[r].name == token)
        {
            found_node = r;
            break;
        }
    }

    if (node_list[found_node].die > d_M) d_M = node_list[found_node].die;
    if (node_list[found_node].die < d_m) d_m = node_list[found_node].die;
}

```

```

    }
    file1.close();
}

void sort_die(die *die_list, int *d_sorting, int n_dies)
{
    int i_d_c_m = -1, d_c_m = 5000;
    int is_sorted[10];
    for (int i = 0; i < n_dies; i++) is_sorted[i] = 0;
    for (int i = n_dies; i > 0; i--)
    {
        for (int j = 0; j < n_dies; j++)
        {
            if (die_list[j].capacity < d_c_m && is_sorted[j] == 0)
            {
                i_d_c_m = j;
                d_c_m = die_list[j].capacity;
            }
        }
        d_sorting[i - 1] = i_d_c_m;
        is_sorted[i_d_c_m] = 1;
    }
}

void update_die_list(node *node_list, die *die_list, int n_dies, int n_nodes)
{
    //clear the die list
    for (int i = 0; i < n_dies; i++)
    {
        for (int j = 0; j < die_list[i].capacity; j++)
        {
            die_list[i].list[j] = -1;
        }
        die_list[i].capacity = 0;
    }

    for (int i = 0; i < n_nodes; i++)
    {
        die_list[node_list[i].die].add_to_die(i);
    }
}

void optimize(string file_name, cell *cell_list, die *die_list, node *node_list, int
n_dies, int n_nets, int n_nodes, float average_cell_using, int bridge)
{
    ifstream file(file_name + ".nets");

```

```

if (!file.is_open())
{
    cout << "Cannot open file: " << file_name + ".nets" << endl;
    return;
}
file.close();
//select the optimization mode
int mode = -1;
float a_c_u;
int b;
check(file_name, cell_list, node_list, die_list, b, a_c_u);
if (b > bridge) mode = 1;
else
{
    if (a_c_u > average_cell_using) mode = 2;
    else mode = 0;
}
if (mode == -1) mode = 0;
//optimization
switch (mode)
{
case 1://optimize the partitioning result based on the limited SLL
{
    //find the net with the largest SLL using
    int d_M = 0, d_m = 3, i_net = 0;
    int n_b_M = 0;
    int i_n_b_M = -1;
    int i_node = 0, n_c_M = 0;
    string line;
    file.open(file_name + ".nets");
    while (getline(file, line))
    {
        if (line.empty()) continue;

        stringstream iss(line);
        string token;
        if (!(iss >> token)) continue;

        if (token == "net") continue;
        if (token == "endnet")
        {
            if (d_M - d_m > n_b_M)
            {

```

```

        if (n_c_M < i_node)
        {
            n_b_M = d_M - d_m;
            i_n_b_M = i_net;
            n_c_M = i_node;
        }
    }
    i_net++;
    i_node = 0;
    d_M = 0;
    d_m = 3;
    continue;
}

int found_node = -1;
for (int r = 0; r < n_nodes; r++)
{
    if (node_list[r].name == token)
    {
        found_node = r;
        break;
    }
}

if (node_list[found_node].die > d_M) d_M = node_list[found_node].die;
if (node_list[found_node].die < d_m) d_m = node_list[found_node].die;
i_node++;
}

file.close();
if (n_b_M == 0) //if the SLL using of every net is 0, then the partitioning
result can't be optimized any more
{
    cout << "Can't optimize the partitioning result any more. " << endl;
    return;
}
else
{
    //move the nodes
    file.open(file_name + ".nets");
    d_M = 0, d_m = 3, i_net = 0;
    while (getline(file, line))
    {
        if (line.empty()) continue;

        stringstream iss(line);

```



```

string token;
if (!(iss >> token)) continue;

if (token == "net") continue;
if (token == "endnet")
{
    if (i_net == i_n_b_M) break;
    else
    {
        i_net++;
        continue;
    }
}

if (i_net == i_n_b_M)
{
    int found_node = -1;
    for (int r = 0; r < n_nodes; r++)
    {
        if (node_list[r].name == token)
        {
            found_node = r;
            break;
        }
    }
    if (node_list[found_node].die > d_M) d_M =
node_list[found_node].die;
    if (node_list[found_node].die < d_m) d_m =
node_list[found_node].die;
}
else continue;
}
file.close();
file.open(file_name + ".nets");
i_net = 0;
while (getline(file, line))
{
    if (line.empty()) continue;

    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;

    if (token == "net") continue;

```

```

        if (token == "endnet")
        {
            if (i_net == i_n_b_M) break;
            else
            {
                i_net++;
                continue;
            }
        }

        if (i_net == i_n_b_M)
        {
            int found_node = -1;
            for (int r = 0; r < n_nodes; r++)
            {
                if (node_list[r].name == token)
                {
                    found_node = r;
                    break;
                }
            }
            if (node_list[found_node].die == d_M)
                node_list[found_node].die = d_M - 2;
            else continue;
        }
        else continue;
    }
    file.close();
}

update_die_list(node_list, die_list, n_dies, n_nodes);
break;
}

case 2://optimize the partitioning result based on the limited average cell using
{
    int d_sort[4];
    sort_die(die_list, d_sort, n_dies);
    int i_net = 0, i_node = 0, i_net_selected = -1;
    int has_die[10], is_in_die[10];
    int net_capacity_M = 0;
    int S = -1;
    for (int i = 0; i < n_dies; i++)
    {
        has_die[i] = 0;
        is_in_die[i] = 0;
    }

```

```

}
int sum_is_in_die;
string line;
//select a net to move its nodes (or the whole net)
while (getline(file, line))
{
    if (line.empty()) continue;

    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;

    if (token == "net") continue;
    if (token == "endnet")
    {
        for (int i = 0; i < n_dies; i++) sum_is_in_die += is_in_die[i];
        if (is_in_die[d_sort[0]] == 0)
        {
            i_net++;
            i_node = 0;
            for (int i = 0; i < n_dies; i++)
            {
                has_die[i] = 0;
                is_in_die[i] = 0;
            }
            sum_is_in_die = 0;
            continue;
        }
    }
    else
    {
        if (sum_is_in_die == 1)
        {
            i_net++;
            i_node = 0;
            for (int i = 0; i < n_dies; i++)
            {
                has_die[i] = 0;
                is_in_die[i] = 0;
            }
            sum_is_in_die = 0;
            continue;
        }
        else
        {

```

```
        if (i_node > net_capacity_M)
        {
            net_capacity_M = i_node;
            i_net_selected = i_net;
            S = 0;
        }
        i_net++;
        i_node = 0;
        for (int i = 0; i < n_dies; i++)
        {
            has_die[i] = 0;
            is_in_die[i] = 0;
        }
        sum_is_in_die = 0;
        continue;
    }
}

int found_node = -1;
for (int r = 0; r < n_nodes; r++)
{
    if (node_list[r].name == token)
    {
        found_node = r;
        break;
    }
}

is_in_die[node_list[found_node].die] = 1;
has_die[node_list[found_node].die]++;
}

file.close();
if (i_net_selected == -1)//special situation
{
    S = 1;
    i_net = 0;
    i_node = 0;
    for (int i = 0; i < n_dies; i++)
    {
        has_die[i] = 0;
        is_in_die[i] = 0;
    }
    file.open(file_name + ".nets");
    while (getline(file, line))
```

```

{
    if (line.empty()) continue;

    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;

    if (token == "net") continue;
    if (token == "endnet")
    {
        if (is_in_die[d_sort[0]] == 1)
        {
            i_net_selected = i_net;
            break;
        }
        else
        {
            i_net++;
            i_node = 0;
            for (int i = 0; i < n_dies; i++)
            {
                has_die[i] = 0;
                is_in_die[i] = 0;
            }
            continue;
        }
    }

    int found_node = -1;
    for (int r = 0; r < n_nodes; r++)
    {
        if (node_list[r].name == token)
        {
            found_node = r;
            break;
        }
    }
    is_in_die[node_list[found_node].die] = 1;
}
file.close();
}
if (S == -1)
{
    cout << "Can't optimize the partitioning result any more. " << endl;
}

```

```
        return;
    }
    //move nodes (or net)
    i_net = 0;
    i_node = 0;
    for (int i = 0; i < n_dies; i++)
    {
        has_die[i] = 0;
        is_in_die[i] = 0;
    }
    file.open(file_name + ".nets");
    while (getline(file, line))
    {
        if (line.empty()) continue;

        stringstream iss(line);
        string token;
        if (!(iss >> token)) continue;

        if (token == "net") continue;
        if (token == "endnet")
        {
            if (i_net != i_net_selected)
            {
                i_net++;
                continue;
            }
            else break;
        }

        if (i_net != i_net_selected) continue;
        else
        {
            int found_node = -1;
            for (int r = 0; r < n_nodes; r++)
            {
                if (node_list[r].name == token)
                {
                    found_node = r;
                    break;
                }
            }
            is_in_die[node_list[found_node].die] = 1;
            has_die[node_list[found_node].die]++;
        }
    }
}
```

```

    }
}
file.close();
i_net = 0;
file.open(file_name + ".nets");
while (getline(file, line))
{
    if (line.empty()) continue;

    stringstream iss(line);
    string token;
    if (!(iss >> token)) continue;

    if (token == "net") continue;
    if (token == "endnet")
    {
        if (i_net != i_net_selected)
        {
            i_net++;
            continue;
        }
        else break;
    }

    if (i_net != i_net_selected) continue;
    else
    {
        int found_node = -1;
        for (int r = 0; r < n_nodes; r++)
        {
            if (node_list[r].name == token)
            {
                found_node = r;
                break;
            }
        }
        if (node_list[found_node].die != d_sort[0]) continue;
        else
        {
            if (S == 1) node_list[found_node].die = d_sort[3];
            else
            {
                if (is_in_die[d_sort[3]] == 1) node_list[found_node].die =
d_sort[3];

```

```

        else if (is_in_die[d_sort[2]] == 1)
node_list[found_node].die = d_sort[2];
        else node_list[found_node].die = d_sort[1];
    }
}
}
}
file.close();
update_die_list(node_list, die_list, n_dies, n_nodes);
break;
}
default:
    return;
}
return;
}

void create_result_file(string file_name, node *node_list, int n_nodes, int n_dies)
{
    ofstream file;
    file.open(file_name + ".hgr.part." + to_string(n_dies) + ".optimized");
    if (!file.is_open())
    {
        cout << "Cannot open file: " << file_name + ".hgr.part." + to_string(n_dies) +
".optimized" << endl;
        return;
    }
    for (int i = 0; i < n_nodes; i++)
    {
        file << node_list[i].die << endl;
    }
    file.close();
}

int main()
{
    int m_s;
    //mode selection:
    //0 for preparation, 1 for partitioning, 2 for placement, 3 for checking, 4 for
optimization
    string file_name;
    die dies[4];
    node nodes[10000];
    cell cells[13];

```



```

int n_c, n_n, n_p, n_no;
int bridge, _b;
float a_cell_using, _a_c_u;
cin >> file_name;
init(file_name, dies, nodes, cells, n_c, n_n, n_p, n_no);
while (true)
{
    cout << "Please select the mode:\n0 for preparation, 1 for partitioning, 2 for
placement, 3 for checking, 4 for optimization, others for exit. " << endl;
    cin >> m_s;
    switch (m_s)
    {
        case 0:
            create_hgr(file_name, nodes, n_no, n_n, n_p);
            break;
        case 1:
            //part(file_name + ".hgr", 4); /*using PaToH*/
            break;
        case 2:
            place(file_name, 4, nodes);
            for (int r = 0; r < n_no; r++)
                dies[nodes[r].die].add_to_die(r);
            break;
        case 3:
            check(file_name, cells, nodes, dies, bridge, a_cell_using);
            cout << "The average cell using is: " << a_cell_using << endl;
            cout << "The SLL using is: " << bridge << endl;
            break;
        case 4:
            cout << "Please input the requested average cell using and SLL: " << endl;
            cin >> _a_c_u >> _b;
            optimize(file_name, cells, dies, nodes, 4, n_n, n_no, _a_c_u, _b);
            create_result_file(file_name, nodes, n_no, 4);
            break;
        default:
            return 0;
    }
}
}

```

## 致谢

首先，感谢郭龙坤老师对我耐心且专业的指导，他在论文写作流程和方法方面对我的教导使我受益匪浅，从而让我顺利完成本次毕业设计（论文）。我认为一篇论文不能代表我在专业方面的水平，更不应该止步于此，而是要学习郭龙坤老师不断学习的精神和勤奋求真的治学态度，在相关领域开拓进取，学以致用，成为该领域的人才，不负郭龙坤老师的谆谆教诲。

其次，感谢给我帮助的所有同学以及学长学姐，他们不仅为我解答了很多我不熟悉的难题，也为我送来相关资料和论文写作技巧，让我体会到同学之间互帮互助、团结奋进的真挚友情，这种温暖、团结、进取的学习气氛，让我感动，让我一生铭记。

最后感谢我的家人，是他们多年来对我学业的支持才让我走到这一步，才使我得以顺利完成学业。